

实践5：基于分层强化学习的人形机器人导航



深蓝学院《人形机器人运动控制》课程第4章

课程链接：<https://www.shenlanxueyuan.com/course/864>

1. 作业目标

本作业基于 Isaac Lab、RSL-RL 与 Unitree G1（29 DoF），实现一个两层分层强化学习导航系统：

- 高层导航策略（需要训练）：根据目标、局部高度感知和机器人状态输出速度指令；
- 低层运动控制策略（已提供并冻结）：将速度指令转换为 29 维关节位置动作。

代码块

1 目标与局部感知

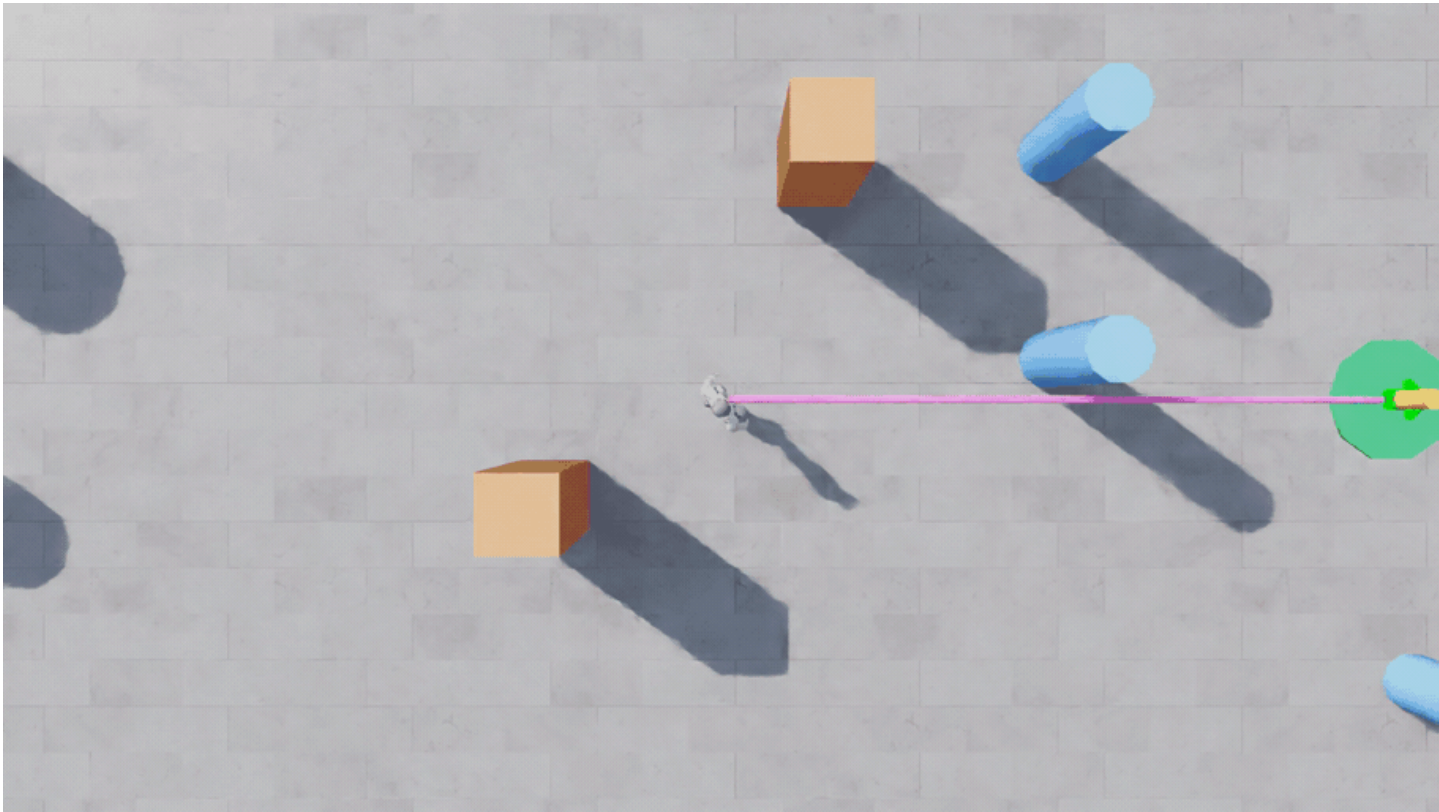
2

3

4 高层 PPO planner $\rightarrow (v_x, v_y, yaw_rate) \rightarrow$ 冻结低层策略 $\rightarrow$ 29维关节动作 $\rightarrow$ G1

5

```
6 pretrained/g1_29dof_lowlevel/policy.pt
```



本作业不要求：

- 重新训练低层运动控制策略；
- 从零设计低层 locomotion reward；
- 重复安装 Isaac Sim、Isaac Lab 或 Unitree G1 模型资产；
- 从零实现一套通用路径规划器。

本作业要求：

1. 完成 7 个代码 TODO，理解高层 observation 与冻结低层 controller 的连接方式；
2. 在静态预构建地图上训练 `HRL-Baseline` ；
3. 理解高层、低层与物理仿真的不同控制频率；
4. 至少选择一个开放方向，将 baseline 推向更难的导航场景；
5. 在相同训练预算下完成定量与定性对比。

本次作业分为2大部分：

- Part 1 基础实现：50 分；
- Part 2 开放探索：50 分。

完成本作业后，你应当具备以下能力：

能力	你应当能够解释或完成
分层建模	为什么导航规划与全身运动控制适合使用不同状态、动作和时间尺度

策略复用	如何将冻结的低层 locomotion policy 接入新的高层任务
接口设计	为什么速度指令是比 29 维关节动作更紧凑、可检查的层间接口
观测设计	高层为何同时需要目标、局部地形、本体状态和控制历史
实验分析	如何在固定预算下比较 baseline 与更难任务，并分析失败模式
课程学习	当地图、障碍或目标难度提高时，何时需要 curriculum learning

2. 作业背景

2.1 为什么使用分层强化学习

直接让单个策略从地图观测输出 29 个关节动作，既要学习避障与目标规划，又要学习平衡和步态，会把长时程决策与高频稳定控制耦合在同一个搜索空间中。分层强化学习（Hierarchical Reinforcement Learning, HRL）利用时间抽象和状态/动作抽象，将复杂任务拆成多个可复用的子问题 [1, 2]：

- 低层策略专注于短时间尺度的稳定行走；
- 高层策略专注于较长时间尺度的目标导航与避障。

在低层技能已经可用时，高层不必重新探索每一种关节级步态，因此通常能减少当前任务中无效的探索，并通过技能复用提高训练效率。需要注意：**HRL 并不无条件保证样本效率更高**；收益依赖于层间接口是否合适、低层技能是否覆盖任务需求，以及预训练成本能否在多个任务间摊销 [1]。

三维速度命令 `(vx, vy, yaw_rate)` 还提供了可解释的层间接口：可以直接检查高层要求机器人往哪里走、命令是否饱和，以及失败来自规划层还是运动层。这类“高层负责语义/子目标，低层负责连续控制”的解耦也广泛出现在分层 VLA 和通才机器人系统中 [4]。它们与本作业并非同一种训练算法，但共享相似的模块化思想。

在机器人导航中，类似的层级设计也可将学习式局部导航器与鲁棒 locomotion controller 连接起来，在复杂地形和障碍环境中分别处理移动决策与运动执行 [5]。

方法	策略输出	优点	主要挑战
端到端单策略	29 维关节动作	接口少、可联合优化	长短时间尺度耦合，探索与归因困难
本作业 HRL	高层 3 维速度；低层 29 维关节目标	复用 locomotion、接口清晰、便于分层调试	受低层能力边界和接口设计约束
经典 Options/技能层级	高层选择 temporally extended action	可组合和复用技能 [3]	技能发现与终止条件设计较难

2.2 与其他实践的关系

- **实践1：宇树 G1 仿真环境搭建与基础功能验证** 已完成 Isaac Sim、Isaac Lab、`unitree_rl_lab` 与 G1 模型资产的安装。本实践可直接复用该环境。
- 低层策略由 `Unitree-G1-29dof-LowLevel` 任务训练。本实践把它视为已经掌握的运动技能，只训练上层导航策略（如果你感兴趣，也可以自己训练更好的底层策略！我们提供的预训练权重训练了4w轮）。

2.3 冻结的低层策略

随作业提供的低层策略位于：

代码块

```
1 pretrained/g1_29dof_lowlevel/  
2 |—— policy.pt           # 导航任务实际加载的 TorchScript 策略  
3 |—— params/              # 训练配置快照
```

导航训练中只进行推理，不对低层策略反向传播，也不更新其参数。

关键约束：

1. 高层速度指令必须裁剪到低层策略熟悉的范围；
2. 推理推荐使用 `torch.inference_mode()`；

其中本体观测采用 5 帧历史。

2.4 理论与代码对应

数据流环节	输入 → 输出	主要代码位置
高层观测	目标、地形、本体状态、控制历史 → actor observation	<code>tasks/navigation/mdp/observations.py</code> ; <code>navigation_env_cfg.py</code>
高层动作桥接	3 维速度动作 → 裁剪后的低层速度命令	<code>tasks/navigation/mdp/pre_trained_policy_action.py</code>
冻结低层推理	低层观测历史 → 29 维关节目标	<code>PreTrainedPolicyAction</code> ; <code>pretrained/g1_29dof_lowlevel/policy.pt</code>
任务与算法配置	scene、command、reward、termination、PPO	<code>navigation_env_cfg.py</code> ; <code>agents/rsl_rl_ppo_cfg.py</code>
训练与评估入口	task ID + 配置 → 日志/checkpoint/可视化	<code>scripts/rsl_rl/train.py</code> ; <code>scripts/rsl_rl/play.py</code>

上述路径均相对于：

代码块

```
1 source/unitree_rl_lab/unitree_rl_lab/
```

2.5 两个导航任务

Task ID	作业角色	地图与目标	Curriculum
Unitree-G1-29dof- Navigation-HRL- Baseline	Part 1 baseline / Part 2 对照组	三种静态预构建地图；每 episode 一个 5-10 m 目 标；进入 0.5 m 成功半径 即结束	无
Unitree-G1-29dof- Navigation-HRL- Extension	开放题参考模板	每次 reset 随机生成混合 障碍；长距离、多目标	障碍数量 curriculum

`HRL-Baseline` 是本作业的 baseline。它并不随机生成新地图：三种模板在环境启动时分配给各并行环境，并在后续 episode 中保持不变。它使用 2×2 max pooling 将 1092 维高度扫描压缩为 273 维，以降低高层网络输入规模。

`HRL-Extension` 不是要求直接完成的第二份作业，而是一个可运行的扩展参考。重点阅读它如何随机布置障碍、如何增加障碍数量，以及如何配置 curriculum。

3. 环境配置

实践1 已安装 Isaac Sim、Isaac Lab、Unitree 模型资产并创建 conda 环境；本实践只安装完整学生作业包中的 Python package, 并做增量更新。若实践1的基础任务尚不能运行，请先回到 实践1 排查，不要在两个仓库之间逐文件覆盖。

3.1 系统要求

- Linux（推荐 Ubuntu 22.04 或更新版本）；
- NVIDIA GPU 与兼容的驱动；
- Isaac Sim 5.1 或更新 (推荐 5.1)；
- Isaac Lab 2.3.2 或更新 (推荐 3.0.0)；
- 能够运行原生 Unitree G1 USD 资产。

3.2 作业包与工作目录

本实践直接使用课程提供的 `unitree_rl_lab_student.zip`。压缩包已经包含完整代码、低层 TorchScript 权重和可选 ONNX 文件，**不需要 Git clone，也不需要 Git LFS**。

先检查压缩包完整性，再解压到一个新建的空目录。以下路径仅为示例：

代码块

```
1  # 1. 检查压缩包；最后应显示 No errors detected
2  unzip -t /path/to/unitree_rl_lab_student.zip
3
4  # 2. 创建新的目标目录。若目录已存在，请换一个新目录，不要覆盖旧作业包
5  mkdir ~/projects/unitree_rl_lab_student
6
7  # 3. 解压。压缩包内部已经包含顶层 unitree_rl_lab/ 目录
8  unzip /path/to/unitree_rl_lab_student.zip -d ~/projects/unitree_rl_lab_student
9
10 # 4. 进入仓库根目录
11 cd ~/projects/unitree_rl_lab_student/unitree_rl_lab
```

正确目录结构如下：

代码块

```
1  unitree_rl_lab_student/unitree_rl_lab/      # 以下命令的工作目录
2  |—— unitree_rl_lab.sh
3  |—— pretrained/g1_29dof_lowlevel/
4  |   |—— policy.pt                          # 已包含在课程压缩包中，训练/play 实际加
   载
5  |   |—— policy.onnx                        # 已包含，可选检查/部署，本作业不加载
6  |   |—— params/
7  |—— source/unitree_rl_lab/unitree_rl_lab/
8  |   |—— tasks/navigation/                 # 7 个 TODO
9  |—— scripts/
10 |—— logs/                                  # 训练后生成，不提交
```

本作业包已经包含匹配的 navigation、locomotion、训练入口和配置；它只复用实践1创建的运行环境。

3.3 增量安装

保持当前目录为含 `unitree_rl_lab.sh` 的作业包根目录，激活实践1创建的 conda 环境，并指向实践1的 Isaac Lab：

代码块

```
1  conda activate env_isaaclab
```

```
2 export ISAACLAB_PATH=~/projects/IsaacLab # 按实践1的实际路径修改
3 ./unitree_rl_lab.sh -i
```

如果实践1使用了其他 conda 环境名或 Isaac Lab 路径，请替换示例值。 `-i` 只增量安装本作业的 Python package；Unitree 模型目录等资产配置沿用实践1。

`unitree_rl_lab.sh` 中可能仍会执行一条兼容 Git 仓库发布方式的 `git lfs install`；课程压缩包运行并不依赖它，因为实际权重已经随 zip 解压。安装是否成功应以 editable package 安装完成及下一节任务注册结果为准。

3.4 快速自检

以下命令均在作业包根目录执行。

Step 1: 确认任务注册。

代码块

```
1 ./unitree_rl_lab.sh -l
```

预期至少包含：

代码块

```
1 Unitree-G1-29dof-LowLevel
2 Unitree-G1-29dof-Navigation-HRL-Baseline
3 Unitree-G1-29dof-Navigation-HRL-Extension
```

Step 2: 确认低层 checkpoint。

代码块

```
1 test -s pretrained/g1_29dof_lowlevel/policy.pt \
2   && echo "low-level policy: OK" \
3   || echo "ERROR: policy.pt missing or empty"
```

课程压缩包中已经包含 `pretrained/g1_29dof_lowlevel/policy.pt`（约 1.7 MB）和 `policy.onnx`。若上一步输出 `ERROR`，说明解压目录不完整、使用了旧目录，或下载的压缩包损坏。不要执行 `git lfs pull`，也不要从网络下载其他模型替代；应重新执行：

代码块

```
1 unzip -t /path/to/unitree_rl_lab_student.zip
```


Step 3: 确认 GPU 与关键依赖。

代码块

```
1 python -c "import torch; print('CUDA:', torch.cuda.is_available()); import isaacsim, rsl_rl; print('imports: OK')"
```

预期看到 `CUDA: True` 与 `imports: OK`。

Step 4: 定位作业 TODO。

代码块

```
1 grep -R -n "HOMEWORK_TODO"
  source/unitree_rl_lab/unitree_rl_lab/tasks/navigation
```

应能定位 7 组 `HOMEWORK_TODO_N_START/END` 标记。开始时导航环境会在未完成的 TODO 处抛出 `NotImplementedError`，这是预期行为；低层任务仍应可以独立运行。

4. MDP 与控制数据流

4.1 高层 action

高层 action 为：

代码块

```
1 [forward_velocity, lateral_velocity, yaw_rate]
```

参考裁剪范围：

代码块

```
1 forward_velocity ∈ [-0.5, 1.0]
2 lateral_velocity ∈ [-0.5, 0.5]
3 yaw_rate         ∈ [-0.5, 0.5]
```

高层输出不能直接写入关节。`PreTrainedPolicyAction` 将它作为低层 observation 中的 `velocity_commands`，再由冻结策略生成关节动作。

4.2 高层 policy observation

参考 baseline 将目标、局部感知、本体状态和控制历史拼接：

Term	维度	作用
base_lin_vel	3	机体线速度
projected_gravity	3	机体姿态/倾斜信息
base_ang_vel	3	机体角速度，scale 0.2
pose_command	4	机器人坐标系下的目标位置与朝向
last_cmd	3	上一条裁剪后的高层指令
height_scan_pooled	273	2×2 max-pooled 局部高度扫描
joint_pos_rel	29	相对关节位置
joint_vel_rel	29	相对关节速度，scale 0.05
low_level_last_action	29	冻结低层策略上一关节动作
合计	376	Compact actor observation

Critic 额外观察 base height 与目标距离。你可以在开放题中尝试其他设计，但 Part 1 首先实现给定 baseline，以便统一验证。376 是当前 Compact 配置的参考值；若你主动修改了观测空间，请在报告中重新给出维度分解。

4.3 时间尺度

仿真、低层策略与高层策略使用不同时间尺度：

层级	配置	周期	频率
物理仿真	sim.dt = 0.005 s	0.005 s	200 Hz
低层策略	low_level_decimation = 4	0.02 s	50 Hz
高层策略	decimation = 4 × 10 = 40	0.2 s	5 Hz

低层动作在两次低层推理之间保持，但底层 joint action term 仍需在每个物理步执行。

5. Part 1：基础实现（50 分）

你需要在 3 个文件中完成 7 个 TODO。不得删除 TODO 边界标记；删除对应的 raise NotImplementedError 后填入实现。

建议按 TODO 6 → 3 → 4 → 5 → 1 → 2 → 7 完成 (请根据自己的代码习惯和阅读习惯动态调整)：先恢复低层输入契约并打通冻结策略执行链，再补齐高层观测。该顺序不是评分要求，但能让

报错更接近根因。

以下区域不是 Part 1 的修改目标：

- `tasks/locomotion/` 中的低层训练任务；
- `pretrained/g1_29dof_lowlevel/` 中的权重和配置快照；
- `scripts/rsl_rl/train.py`、`play.py` 等通用入口。

除非你的 Part 2 方案确有需要且在报告中说明，否则不要修改这些区域。

5.1 文件索引

文件	TODO	内容
<code>tasks/navigation/mdp/observations.py</code>	1-2	高层动作历史与高度扫描池化
<code>tasks/navigation/mdp/pre_trained_policy_action.py</code>	3-5	加载冻结策略、动作裁剪、分层执行循环
<code>tasks/navigation/robots/g1/29dof/navigation_env_cfg.py</code>	6-7	低层推理 observation 与高层 observation 配置

完整路径均位于：

代码块

```
1 source/unitree_rl_lab/unitree_rl_lab/
```

 以下内容是对代码 TODO 区域的补充说明，建议结合代码上下文阅读。

5.2 TODO 1：上一条高层命令（5 分）

函数：`last_high_level_command`

作用：让高层策略观察低层实际接收的速度命令。

- 根据 `action_name` 从 `env.action_manager` 取得对应 action term。
- 返回裁剪后的 `processed_actions`，而不是 `raw_actions`。

5.3 TODO 2：高度扫描池化（8 分）

函数：`height_scan_pooled`

作用：在保留局部障碍空间结构的同时压缩高度扫描。

1. 调用 Isaac Lab `height_scan` 得到扁平高度；
2. 复用同文件的辅助函数，根据 ray pattern 恢复二维网格；
3. 添加 channel 维后执行 `F.max_pool2d`；
4. 将池化结果重新展平。

不要直接对一维向量池化；reshape 顺序必须与 grid pattern 一致。

5.4 TODO 3: 加载并连接冻结低层策略（10 分）

位置: `PreTrainedPolicyAction.__init__`

作用: 连接高层速度命令、低层观测管理器与冻结的 locomotion policy。

- 从 `cfg.policy_path` 加载 TorchScript，将其移动到环境 device 并设为 eval；
- 将低层 observation 的 `velocity_commands` 绑定到裁剪后的高层动作；
- 将 `last_action`（或兼容名称 `actions`）绑定到缓存的低层动作；
- 创建低层 `ObservationManager`。

该策略在导航训练中保持冻结，不创建 optimizer，也不进入训练模式。

5.5 TODO 4: 高层动作裁剪（5 分）

函数: `PreTrainedPolicyAction.process_actions`

作用: 将高层动作限制在低层策略熟悉的速度范围内。

- 完整保存输入到 `_raw_actions`；
- 按预先构造的逐维上下界裁剪，并写入 `_processed_actions`。

不要原地修改输入，否则会同时覆盖用于日志与调试的原始动作。

5.6 TODO 5: 分层动作执行循环（10 分）

函数: `PreTrainedPolicyAction.apply_actions`

作用: 实现高层、低层与物理仿真的多频率控制。

1. 仅在 counter 到达 `low_level_decimation` 周期时更新低层策略；
2. 更新时计算低层 observation，并在 `torch.inference_mode()` 中执行冻结策略；
3. 将输出交给低层 action term 的 `process_actions`；
4. 每个物理步调用低层 action term 的 `apply_actions`，并维护 counter。

低层策略在相邻两次推理之间保持最近一次关节目标；“不重新推理”不等于“不执行动作”。

5.7 TODO 6: 构造低层推理 observation（6 分）

函数: `make_low_level_inference_observations`

作用：从低层训练配置派生与 checkpoint 兼容的推理观测。

- 从低层训练环境的 policy observation 做 `deepcopy`；
- 关闭 corruption，保持 term concatenate 和 5 帧 history；
- 关闭 `base_ang_vel`、`projected_gravity`、`joint_pos_rel`、`joint_vel_rel` 的 noise。

完成时移除 `_raise_homework_todo_6` 占位替换。不要手写或删除 observation term。

5.8 TODO 7：设计高层 planner observation（6 分）

位置：`NavigationObservationsCfg.PolicyCfg`

作用：为高层策略组合目标、本体状态与控制历史。

在基础 `PolicyCfg` 中配置以下 terms：

- `base_lin_vel`、`projected_gravity`、`base_ang_vel`（scale 0.2）；
- `pose_command`、`last_cmd`；
- `joint_pos_rel`、`joint_vel_rel`（scale 0.05）；
- `low_level_last_action`。

高度扫描由 V5 子类配置，基础 `PolicyCfg` 中不要重复添加。保持 corruption 关闭并 concatenate terms；critic 额外使用的 base height 与 command distance 不加入 actor。

6. 训练与验证

6.1 Smoke test

完成全部 TODO 后，先运行：

代码块

```
1  ./unitree_rl_lab.sh -t \  
2  --task Unitree-G1-29dof-Navigation-HRL-Baseline \  
3  --num_envs 16 \  
4  --max_iterations 1
```

Smoke test 只验证环境创建、observation 维度、低层策略加载和一次 PPO update。通过标志是进程完成 1 次 iteration，且没有：

- `HOMEWORK_TODO_N`；
- TorchScript input shape/device 错误；
- observation group 或 command name 错误；

- CUDA out of memory。

它**不证明策略已经学会导航**。若 smoke 通过，可先用相同 seed 做 500–1000 iterations 的短训，确认 reward、成功率和跌倒率能够被记录，再投入完整训练。

6.2 Baseline 完整训练

当前 PPO 配置默认 `max_iterations=30000`、`num_steps_per_env=8`。为便于复现，建议把预算和 seed 显式写入命令：

代码块

```
1  ./unitree_rl_lab.sh -t \  
2  --task Unitree-G1-29dof-Navigation-HRL-Baseline \  
3  --num_envs 4096 \  
4  --seed 42 \  
5  --max_iterations 30000
```

若显存不足，可降低并行环境数量，例如 2048、1024 或 512。环境数量改变后，每次 iteration 收集的 transition 数量也会改变；因此对比实验必须保持相同的 `num_envs`、`num_steps_per_env`、iteration 与随机种子，或改用总 environment steps 作为统一预算。

日志与 checkpoint 默认写入：

代码块

```
1  logs/rsl_rl/unitree_g1_29dof_navigation_hrl_baseline/
```

6.3 Play

代码块

```
1  ./unitree_rl_lab.sh -p \  
2  --task Unitree-G1-29dof-Navigation-HRL-Baseline \  
3  --num_envs 1 \  
4  --checkpoint /path/to/model_x.pt
```

若不显式指定 checkpoint，play 脚本会按其默认规则查找最近一次 run。

建议先检查机器人是否能稳定响应目标速度，再观察是否能绕开障碍并进入目标半径。Play 中偶尔成功不能代替批量评估。

6.4 TensorBoard

代码块

```
1 tensorboard --logdir logs/rsl_rl
```

建议记录和比较：

指标	主要含义
mean reward	总体学习趋势；需结合各 reward term 判断
mean episode length	episode 是否因成功、跌倒或超时提前结束
goal reached / success fraction	进入 0.5 m 成功半径的比例
base height / bad orientation fraction	跌倒或失稳的主要终止原因
command position error	到目标的距离是否持续下降
action rate / action magnitude	高层命令是否剧烈变化或长期饱和
policy mean action std / value loss	探索尺度与 critic 拟合状态

不要只根据 mean reward 选择模型：奖励可能上升，但机器人仍可能通过保守不动、频繁跌倒或命令饱和形成非预期行为。

6.5 Baseline 对照协议

本作业不提供预训练高层 checkpoint。你的 Part 1 训练结果就是对照组。进入 Part 2 前，请固定：

- baseline 配置；
- 随机种子；
- 并行环境数量；
- 总 iteration；
- 评估 episode 数量。

开放题实验只修改你声明的因素，其余条件尽量保持一致。至少保存一组 baseline 曲线和一组 extension 曲线。算力允许时使用 3 个或更多随机种子并报告均值与波动。若只能使用单个 seed，请在报告中明确这一限制，不要把单次结果解释为稳定改进。

6.6 分阶段验收

阶段	最低验收
环境	三个 task ID 可见， <code>policy.pt</code> 存在，CUDA 与 imports 正常

TODO	7 个占位异常均移除；低层输入契约与高层 376 维观测正确
Smoke	1 次 PPO update 完成，无 shape/device/manager 错误
短训	TensorBoard 中 reward、termination 和 command error 可正常记录
完整训练	成功率出现改善趋势，跌倒率和命令误差可解释
Play	机器人能响应目标、保持行走，并在部分地图中完成导航

7. Part 2：开放探索（50 分）

当前 `HRL-Baseline` 只是一个受控 baseline：

- 使用静态预构建地图；
- 只有三种固定模板；
- 每个 episode 只有一个 5-10 m 目标；
- 没有障碍难度 curriculum；
- 障碍不会移动。

请选择至少一个方向 “push the limits”。你可以组合多个方向，但评分更看重实验闭环与分析，而不是修改数量。

`HRL-Extension` 可作为配置参考，也可直接作为扩展实验起点：

代码块

```
1  ./unitree_rl_lab.sh -t \  
2  --task Unitree-G1-29dof-Navigation-HRL-Extension \  
3  --num_envs 2048 \  
4  --seed 42
```

它比 Baseline 更耗显存，`num_envs` 应按硬件调整。若以该任务作为实验组，仍需明确你实际修改了什么，不能把仓库已有 Extension 配置本身当作开放题贡献。

7.1 方向 A：更难的静态地图

修改：

代码块

可增加：

- 更高障碍密度；
- 更窄通道；
- U形、死胡同或遮挡结构；
- 需要绕行而不能只朝目标直走的布局。

要求说明地图难点，并给出至少一张俯视图或 play 录像。

7.2 方向 B：随机障碍生成

参考 `HRL-Extension` 任务实现：

- `NavigationV5MixedObstacleEventCfg`；
- `randomize_mixed_obstacle_layout`；
- `MixedObstacleLayoutCfg`。

配置类位于 `navigation_env_cfg.py`，具体布局逻辑位于

`tasks/navigation/mdp/events/` 与 `tasks/navigation/mdp/obstacles/`。将 startup 时固定分配地图改为 reset 时随机生成，比较训练分布和测试分布上的泛化表现。请明确 random seed、障碍数量和几何尺寸范围。

7.3 方向 C：动态移动障碍

可为 kinematic obstacles 添加 interval event 或逐步更新逻辑。建议先从少量、低速、可预测轨迹开始，再增加随机性。

推荐处理：

- 移动障碍不得瞬移到机器人内部；
- height scanner 的 mesh transform 必须保持更新；
- 碰撞与 GPU buffer 开销；
- 训练时障碍速度范围和测试时范围。

仅把障碍移动起来但未训练出可比较策略，不能获得完整实现分。

7.4 方向 D：更长目标或多阶段目标

当前目标距离为 5–10 m。可尝试：

- 10–20 m 或更长距离；
- 到达一个 waypoint 后生成下一个目标；

- 需要穿越多个区域的目标序列。

目标变远时，应同步检查 episode length；否则很多 episode 会因时间不足而结束。

7.5 方向 E：Curriculum learning

当障碍更密、地图随机或目标更长时，直接从最难分布训练可能失败。参考 `HRL-Extension` 任务实现：

代码块

```
1 NavigationV5MixedObstacleCurriculumCfg
2 V5_OBSTACLE_COUNT_LEVELS = (0, 50, 80, 100, 120)
3 tasks/navigation/mdp/curriculums/obstacle_count.py
```

你可以设计：

- 障碍数量 curriculum；
- 障碍速度 curriculum；
- 目标距离 curriculum；
- 通道宽度 curriculum；
- 根据成功率而非固定 iteration 升级的 curriculum。

报告中必须给出升级规则，并解释 curriculum 是否真正改善最终难度下的表现。

7.6 开放题评分（50 分）

项目	分值	要求
问题定义与实现	20	修改可运行；难度提升清楚；代码结构合理
实验与定量对比	15	与 Part 1 baseline 使用可比预算；报告成功率、终止原因和训练曲线
分析与定性结果	15	解释失败模式、方法局限；提供 play 截图/视频；讨论 curriculum 或其他改进

最低实验闭环：

1. 写清唯一主要变量与假设；
2. 使用可比预算训练 baseline 和 extension；
3. 在相同评估协议下报告成功率、目标误差、跌倒/超时比例；

4. 提供训练曲线与至少一段定性演示；
5. 分析至少一种失败模式，而不仅是报告最终分数。

8. 常见问题

8.1 `NotImplementedError: HOMEWORK_TODO_N`

对应 TODO 尚未完成，或答案仍在标记之外。使用 `grep -R -n "HOMEWORK_TODO"` 定位。

8.2 TorchScript 输入维度错误

优先检查 TODO 6：

- 是否使用 `deepcopy` ；
- `history` 是否为 5；
- `term` 顺序是否来自低层训练配置；
- 是否错误删除了 `last action` 或 `velocity command`。
- 是否仍保留 `_raise_homework_todo_6` 占位函数。

8.3 高层动作饱和

检查 TODO 4 是否正确逐维裁剪，并在 TensorBoard 或 debug visualization 中比较 `raw` 与 `processed action`。

8.4 训练看似正常但机器人不走

检查：

- TODO 3 的 `velocity command` 是否指向 `_processed_actions` ；
- TODO 5 是否调用低层 `process_actions` ；
- `counter` 是否更新；
- `checkpoint` 路径是否正确。

还应确认实践1中的 G1 基础任务和资产可以运行，并重新检查本作业的低层 `checkpoint` 路径，以区分基础环境问题与高层桥接问题。

8.5 Baseline observation 不是 376 维

检查 TODO 7 是否同时加入了原始 `height_scan` 和 `height_scan_pooled`。Baseline 的 Compact 子类应只保留 273 维池化项；同时检查是否遗漏 29 维低层上一动作或重复加入 `critic-only terms`。

8.6 随机或动态地图显存不足

减少 `num_envs` 或最大障碍数量。大量 kinematic rigid objects 会显著增加 broadphase pair 与 contact buffer 使用量。

9. 实验报告

报告建议包含：

1. 分层控制结构与物理、高层、低层三个更新频率；
2. Part 1 observation 设计与 376 维分解；
3. 冻结低层策略如何加载与连接；
4. baseline 训练配置、总 environment steps、seed 与结果；
5. Part 2 问题定义、代码修改和难度设置；
6. baseline 与 extension 的定量曲线和统一评估协议；
7. play 的定性结果；
8. 失败模式、局限与下一步改进。

10. 评分检查清单

Part 1（50 分）

- 7 个 TODO 全部完成，边界标记保留；
- HRL-Baseline smoke test 能完成 PPO update；
- 低层模型处于 eval，推理无梯度；
- Compact actor observation 为 376 维；
- baseline 可以训练并 play；
- `Unitree-G1-29dof-LowLevel` 未被作业修改破坏。

Part 2（50 分）

- 至少完成一个难度扩展；
- 有可复现的训练配置；
- 有公平的 baseline 对照；
- 有定量指标与定性演示；
- 分析了失败模式；
- 若任务明显变难，讨论或实现了 curriculum。

11. 提交说明

提交一个压缩包：

代码块

```
1  用户名-实践5.zip
2  |—— code/
3  |   |—— 你修改过的源文件（保持仓库相对路径）
4  |—— report/
5  |   |—— HW5_report_用户名.pdf
6  |   |—— figures/
7  |—— README.txt
```

README.txt 应说明：

- 选择的开放方向；
- 训练与 play 命令；
- 主要配置改动；
- 最佳 checkpoint 的相对路径；
- 复现实验所需的随机种子。

不要提交整个 Isaac Sim、完整 logs/ 或无关依赖。若 checkpoint 过大，请按课程平台要求单独提交或提供下载方式。

12. 延伸阅读

1. “Hierarchical Reinforcement Learning: A Survey and Open Research Challenges,” *Machine Learning and Knowledge Extraction*, 2022. <https://doi.org/10.3390/make4010009>
2. O. Nachum, S. Gu, H. Lee, and S. Levine, “Data-Efficient Hierarchical Reinforcement Learning,” *NeurIPS*, 2018. <https://proceedings.neurips.cc/paper/2018/hash/e6384711491713d29bc63fc5eeb5ba4f-Abstract.html>
3. R. S. Sutton, D. Precup, and S. Singh, “Between MDPs and Semi-MDPs: A Framework for Temporal Abstraction in Reinforcement Learning,” *Artificial Intelligence*, 1999. [https://doi.org/10.1016/S0004-3702\(99\)00052-1](https://doi.org/10.1016/S0004-3702(99)00052-1)
4. Hi Robot: Open-Ended Instruction Following with Hierarchical Vision-Language-Action Models, 2025. <https://arxiv.org/abs/2502.19417>
5. J. Lee et al., “Learning Robust Autonomous Navigation and Locomotion for Wheeled-Legged Robots,” *Science Robotics*, 2024. <https://doi.org/10.1126/scirobotics.adi9641>

